

# LAPORAN TUGAS

Nama = Nabil Faza Al-Musyaffa  
Kelas/NIM = 2024B/25091397054

## Struktur Data Queue (Antrian)

### 1. Tujuan

Laporan ini disusun dengan tujuan sebagai berikut:

- Memahami konsep dan cara kerja struktur data Queue (Antrian).
- Menentukan kompleksitas waktu (worst case) dari setiap operasi Queue.
- Melakukan eksekusi manual terhadap operasi enqueue dan dequeue.
- Mengimplementasikan Queue dalam bahasa pemrograman Python.
- Menjelaskan konversi satuan waktu simulasi dari menit ke detik.
- Membuat fungsi untuk membalik isi Queue menggunakan operasi standar.

### 2. Dasar Teori

#### 2.1 Pengertian Queue

Queue (Antrian) adalah struktur data linear yang mengikuti prinsip FIFO (First In, First Out), yaitu elemen yang pertama kali dimasukkan akan menjadi elemen yang pertama kali dikeluarkan. Queue memiliki dua ujung, yaitu front (depan) sebagai titik pengeluaran data dan rear (belakang) sebagai titik penambahan data.

#### 2.2 Operasi Dasar Queue

Queue memiliki empat operasi utama, yaitu:

| Operasi    | Keterangan                                           | Contoh                     |
|------------|------------------------------------------------------|----------------------------|
| enqueue(x) | Menambahkan elemen x ke bagian rear queue.           | enqueue(5) → queue: [5]    |
| dequeue()  | Menghapus dan mengembalikan elemen dari front queue. | dequeue() → mengeluarkan 5 |
| isEmpty()  | Mengecek apakah queue kosong.                        | isEmpty() → True/False     |
| size()     | Mengembalikan jumlah elemen dalam queue.             | size() → 3                 |

## 2.3 Kompleksitas Waktu Queue

Kompleksitas waktu menggambarkan performa algoritma berdasarkan ukuran input ( $n$ ). Berikut adalah kompleksitas worst case dari setiap operasi Queue:

| Operasi    | Kompleksitas | Alasan                                                                  |
|------------|--------------|-------------------------------------------------------------------------|
| enqueue(x) | $O(1)$       | Penambahan dilakukan langsung di rear tanpa iterasi.                    |
| dequeue()  | $O(1)$       | Penghapusan dilakukan langsung dari front tanpa iterasi.                |
| isEmpty()  | $O(1)$       | Hanya memeriksa panjang list, tidak bergantung pada $n$ .               |
| size()     | $O(1)$       | Python menyimpan panjang list secara internal, sehingga akses langsung. |

Semua operasi dasar Queue memiliki kompleksitas  $O(1)$  (Constant Time), artinya waktu eksekusi tidak bergantung pada jumlah elemen dalam queue.

## 3. Pembahasan

### 3.1 Kompleksitas Waktu Operasi Queue (Worst Case)

Berdasarkan analisis pada Dasar Teori (Tabel 2.3), seluruh operasi Queue memiliki kompleksitas waktu  $O(1)$ . Hal ini dikarenakan implementasi Queue menggunakan list Python yang mendukung akses langsung ke posisi tertentu tanpa perlu melakukan perulangan.

### 3.2 Eksekusi Manual – Hanya Enqueue

Kode yang dieksekusi:

```
for i in range(16):  
    if i % 3 == 0:  
        enqueue(i)
```

Penjelasan: Iterasi dilakukan dari  $i = 0$  hingga  $i = 15$ . Operasi enqueue hanya dijalankan apabila nilai  $i$  habis dibagi 3 ( $i \% 3 == 0$ ).

| i | $i \% 3$     | Aksi       | Isi Queue |
|---|--------------|------------|-----------|
| 0 | $0 \% 3 = 0$ | enqueue(0) | [0]       |
| 1 | $1 \% 3 = 1$ | –          | [0]       |
| 2 | $2 \% 3 = 2$ | –          | [0]       |
| 3 | $3 \% 3 = 0$ | enqueue(3) | [0, 3]    |
| 4 | $4 \% 3 = 1$ | –          | [0, 3]    |
| 5 | $5 \% 3 = 2$ | –          | [0, 3]    |
| 6 | $6 \% 3 = 0$ | enqueue(6) | [0, 3, 6] |

| i  | i % 3      | Aksi        | Isi Queue            |
|----|------------|-------------|----------------------|
| 7  | 7 % 3 = 1  | –           | [0, 3, 6]            |
| 8  | 8 % 3 = 2  | –           | [0, 3, 6]            |
| 9  | 9 % 3 = 0  | enqueue(9)  | [0, 3, 6, 9]         |
| 10 | 10 % 3 = 1 | –           | [0, 3, 6, 9]         |
| 11 | 11 % 3 = 2 | –           | [0, 3, 6, 9]         |
| 12 | 12 % 3 = 0 | enqueue(12) | [0, 3, 6, 9, 12]     |
| 13 | 13 % 3 = 1 | –           | [0, 3, 6, 9, 12]     |
| 14 | 14 % 3 = 2 | –           | [0, 3, 6, 9, 12]     |
| 15 | 15 % 3 = 0 | enqueue(15) | [0, 3, 6, 9, 12, 15] |

Hasil akhir queue setelah eksekusi:

**Queue: [0, 3, 6, 9, 12, 15]**

### 3.3 Eksekusi Manual – Enqueue dan Dequeue

Kode yang dieksekusi:

```
for i in range(16):
    if i % 3 == 0:
        enqueue(i)
    elif i % 4 == 0:
        dequeue()
```

Penjelasan: Pada setiap iterasi, kondisi  $i \% 3 == 0$  diperiksa terlebih dahulu. Jika tidak terpenuhi, kondisi  $i \% 4 == 0$  diperiksa untuk melakukan dequeue. Kedua kondisi bersifat elif (tidak terjadi bersamaan).

| i | i%3 | i%4 | Aksi             | Isi Queue |
|---|-----|-----|------------------|-----------|
| 0 | =0  | =0  | enqueue(0)       | [0]       |
| 1 | =1  | =1  | –                | [0]       |
| 2 | =2  | =2  | –                | [0]       |
| 3 | =0  | =3  | enqueue(3)       | [0, 3]    |
| 4 | =1  | =0  | dequeue()<br>→ 0 | [3]       |
| 5 | =2  | =1  | –                | [3]       |
| 6 | =0  | =2  | enqueue(6)       | [3, 6]    |
| 7 | =1  | =3  | –                | [3, 6]    |
| 8 | =2  | =0  | dequeue()        | [6]       |

| i  | i%3 | i%4 | Aksi        | Isi Queue      |
|----|-----|-----|-------------|----------------|
|    |     |     | → 3         |                |
| 9  | =0  | =1  | enqueue(9)  | [6, 9]         |
| 10 | =1  | =2  | –           | [6, 9]         |
| 11 | =2  | =3  | –           | [6, 9]         |
| 12 | =0  | =0  | enqueue(12) | [6, 9, 12]     |
| 13 | =1  | =1  | –           | [6, 9, 12]     |
| 14 | =2  | =2  | –           | [6, 9, 12]     |
| 15 | =0  | =3  | enqueue(15) | [6, 9, 12, 15] |

Hasil akhir queue setelah eksekusi:

**Queue: [6, 9, 12, 15]**

### 3.4 Implementasi Queue dalam Python

Berikut adalah implementasi sederhana Queue menggunakan class dalam Python:

```
class Queue:
    def __init__(self):
        """Inisialisasi queue kosong."""
        self.items = []
```

```
    def enqueue(self, item):
        """Menambahkan elemen ke bagian rear queue."""
        self.items.append(item)
```

```
    def dequeue(self):
        """Menghapus dan mengembalikan elemen dari front queue."""
        if self.isEmpty():
            return "Queue kosong"
        return self.items.pop(0)
```

```
    def isEmpty(self):
        """Mengecek apakah queue kosong."""
        return len(self.items) == 0
```

```
    def size(self):
        """Mengembalikan jumlah elemen dalam queue."""
        return len(self.items)
```

```
    def __str__(self):
        """Menampilkan isi queue."""
```

```

        return str(self.items)

# === Contoh Penggunaan ===
q = Queue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
print("Queue:", q)          # Output: Queue: [10, 20, 30]
print("Size:", q.size())    # Output: Size: 3
q.dequeue()
print("Setelah dequeue:", q) # Output: Setelah dequeue: [20, 30]

```

### 3.5 Konversi Satuan Waktu Simulasi: Menit ke Detik

Dalam simulasi antrian, satuan waktu menentukan granularitas (tingkat kedetailan) dari proses. Untuk mengubah simulasi dari satuan menit menjadi detik, langkah-langkah berikut perlu dilakukan:

#### a. Ubah Variabel Waktu

Setiap variabel yang menyimpan waktu dalam satuan menit dikonversi menjadi detik dengan mengalikannya dengan 60.

```

# Sebelum (menit)
SIMULATION_TIME = 60    # 60 menit
ARRIVAL_RATE     = 2     # 2 pelanggan per menit

# Sesudah (detik)
SIMULATION_TIME = 3600   # 3600 detik = 60 menit
ARRIVAL_RATE    = 2/60   # ~0.033 pelanggan per detik

```

#### b. Ubah Durasi Layanan

Waktu layanan per pelanggan juga disesuaikan dari menit ke detik.

```

# Sebelum (menit)
SERVICE_TIME = 5        # 5 menit per pelanggan

# Sesudah (detik)
SERVICE_TIME = 300      # 300 detik = 5 menit

```

#### c. Sesuaikan Tick Simulasi

Jika simulasi menggunakan loop berbasis tick (satuan waktu terkecil), setiap tick diubah menjadi representasi 1 detik.

```

# Setiap iterasi merepresentasikan 1 detik
for tick in range(SIMULATION_TIME): # range(3600)

```

```
# proses tiba dan layanan
```

Dengan perubahan di atas, resolusi simulasi meningkat dari 1 menit per iterasi menjadi 1 detik per iterasi, sehingga menghasilkan hasil yang lebih akurat.

### 3.6 Fungsi Membalik Isi Queue

Queue hanya mendukung operasi enqueue dan dequeue. Untuk membalik isi queue, digunakan struktur data tambahan berupa Stack (tumpukan) sebagai media penyimpanan sementara.

#### Prinsip Kerja:

- Keluarkan semua elemen dari queue dan masukkan ke dalam stack.
- Stack bersifat LIFO (Last In, First Out), sehingga urutan elemen otomatis terbalik.
- Keluarkan semua elemen dari stack dan masukkan kembali ke queue.

#### Implementasi Python:

```
def reverse_queue(queue):  
    """  
    Membalik isi queue menggunakan stack sebagai media sementara.  
    Parameter : queue (objek Queue)  
    Return    : queue dengan isi yang sudah dibalik  
    """  
    stack = []
```

```
# Langkah 1: Pindahkan semua elemen queue ke stack  
while not queue.isEmpty():  
    stack.append(queue.dequeue())
```

```
# Langkah 2: Pindahkan kembali dari stack ke queue  
# Stack bersifat LIFO, sehingga urutan menjadi terbalik  
while stack:  
    queue.enqueue(stack.pop())
```

```
return queue
```

```
# === Contoh Penggunaan ===  
q = Queue()  
for val in [10, 20, 30, 40, 50]:  
    q.enqueue(val)
```

```
print("Sebelum dibalik:", q)    # [10, 20, 30, 40, 50]  
reverse_queue(q)  
print("Sesudah dibalik :", q)   # [50, 40, 30, 20, 10]
```

### Trace Eksekusi:

| Langkah | Aksi                       | Stack            | Queue            |
|---------|----------------------------|------------------|------------------|
| Awal    | –                          | []               | [10,20,30,40,50] |
| 1       | dequeue → stack.append(10) | [10]             | [20,30,40,50]    |
| 2       | dequeue → stack.append(20) | [10,20]          | [30,40,50]       |
| 3       | dequeue → stack.append(30) | [10,20,30]       | [40,50]          |
| 4       | dequeue → stack.append(40) | [10,20,30,40]    | [50]             |
| 5       | dequeue → stack.append(50) | [10,20,30,40,50] | []               |
| 6       | stack.pop() → enqueue(50)  | [10,20,30,40]    | [50]             |
| 7       | stack.pop() → enqueue(40)  | [10,20,30]       | [50,40]          |
| 8       | stack.pop() → enqueue(30)  | [10,20]          | [50,40,30]       |
| 9       | stack.pop() → enqueue(20)  | [10]             | [50,40,30,20]    |
| 10      | stack.pop() → enqueue(10)  | []               | [50,40,30,20,10] |

## 4. Kesimpulan

Berdasarkan pembahasan di atas, dapat disimpulkan beberapa hal berikut:

1. Queue adalah struktur data linear bertipe FIFO (First In, First Out). Elemen yang pertama masuk akan menjadi elemen yang pertama keluar.
2. Seluruh operasi dasar Queue (enqueue, dequeue, isEmpty, size) memiliki kompleksitas waktu  $O(1)$ , sehingga sangat efisien untuk digunakan.
3. Eksekusi manual menunjukkan bahwa kombinasi kondisi  $i \% 3 == 0$  dan  $i \% 4 == 0$  bersifat mutually exclusive (tidak dapat terjadi bersamaan selain pada  $i = 0$ ), sehingga alur eksekusi dapat diprediksi dengan jelas.
4. Implementasi Queue dalam Python dapat dilakukan menggunakan class dengan list sebagai penyimpanan internal.
5. Konversi simulasi dari menit ke detik dilakukan dengan mengalikan semua parameter waktu dengan 60, dan menyesuaikan resolusi tick simulasi.
6. Pembalikan isi Queue dapat dilakukan secara efisien menggunakan Stack sebagai struktur data pembantu. Kombinasi FIFO dan LIFO menghasilkan urutan elemen yang terbalik.